

UNIVERSIDAD NACIONAL DE LOJA

Facultad de la Energía, las Industrias y los Recursos Naturales No Renovables

Carrera de Computación

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

(SRS - IEEE 830 Abreviado)

Asignación eficiente de patrullas ante alarmas bancarias en Loja

Asignatura:	Estructura de Datos
Código:	E2C3A1
Ciclo:	3
Período Académico:	Septiembre 2026 – Marzo 2027
Modalidad:	Presencial
Docente:	Ing. Narváez Guillén Cristian Ramiro
Correo:	cristian.narvaez@unl.edu.ec
Actividad:	AA (Semana 01) – Investigación documental y especificación de requerimientos
Versión:	1.0
Fecha:	7 de octubre de 2026

Integrantes del Grupo

Jeancarlos Fernando Aguirre Romero

Willan Sebastián Granda Camacho

Lenin Fabricio Macas Cabrera

Christopher Alexander Pineda Rodas

Eduardo José Soto Cabrera

Control del Documento

Título	Especificación de Requisitos de Software: Asignación eficiente de patrullas ante alarmas bancarias en Loja
Asignatura	Estructura de Datos
Docente	Ing. Narváez Guillén Cristian Ramiro
Versión	1.0
Estado	Final
Fecha	7 de octubre de 2026

Historial de Revisiones

Versión	Fecha	Descripción	Autores
1.0	7 de octubre de 2026	Versión inicial completa del documento SRS	Grupo de trabajo

Índice

1. Introducción	3
1.1. Propósito	3
1.2. Alcance	3
1.3. Definiciones, Acrónimos y Abreviaturas	3
1.4. Referencias	4
2. Descripción General	4
2.1. Perspectiva del Producto	4
2.2. Funciones del Producto	4
2.3. Características de los usuarios	5
3. Requisitos Específicos	5
3.1. Requisitos Funcionales	5
3.1.1. RF-01: Gestión de datos estáticos	5
3.1.2. RF-02: Gestión de datos dinámicos	5
3.1.3. RF-03: Procesamiento y cálculo de rutas	6
3.1.4. RF-04: Actualización dinámica de rutas	6
3.1.5. RF-05: Interfaz y visualización	6
3.1.6. RF-06: Backend	6
3.2. Requisitos No Funcionales	6
3.2.1. RNF-01: Rendimiento	6
3.2.2. RNF-02: Actualización dinámica	7
3.2.3. RNF-03: Usabilidad	7
3.2.4. RNF-04: Escalabilidad	7
3.2.5. RNF-05: Gestión de memoria	7
4. Fundamento Técnico: Gestión de Memoria Estática y Dinámica en V8	7
4.1. Call Stack (Memoria Estática)	7
4.2. Memory Heap (Memoria Dinámica)	7
4.3. Relación con el proyecto	8
A. Anexo A: Comparativa Call Stack vs Memory Heap	8
B. Anexo B: Resumen del Garbage Collection en V8	8
C. Anexo C: Fuentes Consultadas	8

1 Introducción

1.1 Propósito

El presente documento define los requisitos funcionales y no funcionales del sistema “**Asignación eficiente de patrullas ante alarmas bancarias en Loja**”. El sistema tiene como objetivo determinar de manera rápida qué patrulla debe atender una alarma bancaria y cuál es la ruta más eficiente, considerando las condiciones actuales de circulación en el casco urbano de Loja.

Este documento se elabora en el marco de la actividad autónoma de la Semana 01 de la asignatura Estructura de Datos, integrando la investigación técnica sobre gestión de memoria estática (Call Stack) y dinámica (Memory Heap) en el motor V8 de JavaScript.

1.2 Alcance

El sistema deberá:

- Gestionar datos estáticos de la red vial (vías, intersecciones, distancias, bancos y patrullas).
- Gestionar datos dinámicos (alarmas, ubicación y estado de patrullas, condiciones de tráfico).
- Representar la red vial mediante un grafo dirigido implementado con lista de adyacencia.
- Aplicar el algoritmo de Dijkstra con Cola de Prioridad (Min-Heap) para encontrar la ruta de menor costo.
- Seleccionar la patrulla con el menor tiempo estimado de llegada.
- Actualizar dinámicamente los pesos de las vías según las condiciones de tráfico.
- Visualizar la información en un mapa interactivo.

1.3 Definiciones, Acrónimos y Abreviaturas

- **V8**: Motor de ejecución de JavaScript desarrollado por Google.
- **Call Stack**: Pila de ejecución donde se almacenan contextos y valores primitivos.
- **Memory Heap**: Montón de memoria dinámica donde residen objetos, arreglos y estructuras complejas.
- **GC**: Garbage Collector (recolector de basura).

- **Lista de adyacencia:** Estructura de datos para representar grafos.
- **Dijkstra:** Algoritmo para encontrar el camino de menor costo en un grafo.
- **Min-Heap:** Estructura de datos utilizada como Cola de Prioridad.
- **SRS:** Software Requirements Specification.

1.4 Referencias

- MDN Web Docs – Gestión de Memoria en JavaScript.
- Documentación oficial de V8 – Memory Management.
- IEEE Std 830-1998 – Recommended Practice for Software Requirements Specifications.
- Sílabo de Estructura de Datos – Universidad Nacional de Loja (2026).

2 Descripción General

2.1 Perspectiva del Producto

El sistema se orienta a la atención eficiente de emergencias bancarias en el casco urbano de Loja. Combina la gestión de datos estáticos y dinámicos, estructuras de datos (grafos, listas de adyacencia y colas de prioridad) y algoritmos de optimización (Dijkstra), implementados sobre Node.js y visualizados con React y Leaflet.js.

La investigación previa sobre el motor V8 aporta el fundamento teórico de cómo se gestionan en memoria los datos estáticos (asociados preferentemente a la Call Stack) y los datos dinámicos (almacenados en el Memory Heap).

2.2 Funciones del Producto

1. Registrar y gestionar patrullas (ubicación y estado).
2. Registrar alarmas bancarias.
3. Representar la red vial como grafo dirigido con lista de adyacencia.
4. Gestionar y actualizar pesos de las vías (tiempos de recorrido).
5. Calcular rutas óptimas mediante Dijkstra + Min-Heap.
6. Seleccionar la patrulla con menor tiempo estimado de llegada.

7. Actualizar rutas ante cambios en el tráfico.
8. Visualizar patrullas, alarmas y rutas en un mapa interactivo.

2.3 Características de los usuarios

- Personal de seguridad o central de monitoreo que necesita asignar patrullas de forma rápida.
- Desarrolladores y estudiantes que implementan y mantienen el sistema.

3 Requisitos Específicos

3.1 Requisitos Funcionales

3.1.1 RF-01: Gestión de datos estáticos

El sistema deberá gestionar la información básica de la red vial y los recursos:

- Vías y calles del casco urbano de Loja.
- Intersecciones (nodos).
- Distancias o tiempos estimados iniciales entre nodos.
- Ubicación de bancos (posibles puntos de alarma).
- Información básica de las patrullas (identificador, ubicación inicial).
- Representación de la red vial mediante un grafo dirigido implementado con lista de adyacencia.

La lista de adyacencia almacenará, para cada nodo, las conexiones existentes y el peso correspondiente (distancia o tiempo estimado).

3.1.2 RF-02: Gestión de datos dinámicos

El sistema deberá gestionar información que puede cambiar durante la ejecución:

- Registro de alarmas bancarias (ubicación, hora y estado).
- Ubicación actual y estado de las patrullas (disponible, atendiendo emergencia, fuera de servicio).
- Condiciones de tráfico vehicular que modifican los tiempos de recorrido de las vías.

3.1.3 RF-03: Procesamiento y cálculo de rutas

- Representar la red vial como grafo dirigido con lista de adyacencia.
- Aplicar el algoritmo de Dijkstra para encontrar el camino de menor costo entre la ubicación de una patrulla y el punto de la alarma.
- Utilizar una Cola de Prioridad basada en Min-Heap para optimizar la selección de nodos durante la ejecución de Dijkstra.
- Comparar las patrullas disponibles y seleccionar aquella con el menor tiempo estimado de llegada (no necesariamente la más cercana geográficamente).

3.1.4 RF-04: Actualización dinámica de rutas

El sistema deberá permitir la actualización de los pesos de las conexiones del grafo cuando existan cambios relevantes en el tráfico. Tras una actualización significativa, se podrá volver a ejecutar el cálculo de rutas para determinar si existe una alternativa más eficiente.

3.1.5 RF-05: Interfaz y visualización

- Desarrollar la interfaz de usuario con React.
- Representar la información geográfica sobre un mapa interactivo utilizando Leaflet.js.
- Mostrar en el mapa: patrullas disponibles, alarmas generadas y rutas calculadas.
- Presentar métricas de respuesta (distancia estimada y tiempo aproximado de llegada).

3.1.6 RF-06: Backend

El backend será desarrollado con Node.js y Express, siendo responsable de:

- Recibir solicitudes.
- Administrar la información estática y dinámica.
- Ejecutar los algoritmos de selección de patrulla y cálculo de rutas.

3.2 Requisitos No Funcionales

3.2.1 RNF-01: Rendimiento

El cálculo de rutas y la selección de patrulla deben realizarse en un tiempo reducido, adecuado para escenarios de emergencia.

3.2.2 RNF-02: Actualización dinámica

El sistema debe responder a cambios en el tráfico y a la generación de nuevas alarmas de forma oportuna.

3.2.3 RNF-03: Usabilidad

La información presentada en el mapa y en la interfaz debe ser clara y de fácil interpretación.

3.2.4 RNF-04: Escalabilidad

El sistema debe permitir la incorporación posterior de más vías, patrullas, bancos o zonas de la ciudad.

3.2.5 RNF-05: Gestión de memoria

Dado el uso de Node.js (motor V8), el sistema deberá manejar adecuadamente:

- Datos estáticos (estructuras de grafo y configuraciones) que permanecen en memoria durante la ejecución.
- Datos dinámicos (alarmas, estados de patrullas y pesos de tráfico) que se crean, actualizan y liberan frecuentemente, aprovechando el Garbage Collector de V8.

4 Fundamento Técnico: Gestión de Memoria Estática y Dinámica en V8

4.1 Call Stack (Memoria Estática)

La Pila de Ejecución almacena contextos de ejecución y valores primitivos. Su gestión es automática e inmediata al finalizar cada función. Tiene tamaño limitado (riesgo de Stack Overflow).

4.2 Memory Heap (Memoria Dinámica)

El Montón almacena objetos, arreglos, funciones y estructuras complejas (como la lista de adyacencia del grafo, las colas de prioridad y los objetos de alarmas/patrullas). Su liberación es gestionada automáticamente por el Garbage Collector de V8 mediante el algoritmo Mark-and-Sweep y recolección generacional (Young Generation y Old Generation).

4.3 Relación con el proyecto

- Los datos estáticos de la red vial (nodos, conexiones iniciales) se mantienen como estructuras de larga duración en el Heap.
- Los datos dinámicos (alarmas, estados de patrullas, pesos actualizados) se crean y destruyen con mayor frecuencia, por lo que el comportamiento del Garbage Collector es relevante para el rendimiento del sistema.

A Anexo A: Comparativa Call Stack vs Memory Heap

Característica	Call Stack	Memory Heap
Tipo de datos	Primitivos y referencias	Objetos, arreglos, estructuras complejas
Asignación	Estática / tamaño fijo	Dinámica / tamaño variable
Velocidad de acceso	Muy rápida (LIFO)	Relativamente más lenta
Capacidad	Limitada	Mayor (limitada por RAM)
Liberación	Inmediata	Asíncrona (Garbage Collector)

Cuadro 1: Comparativa entre Call Stack y Memory Heap

B Anexo B: Resumen del Garbage Collection en V8

1. **Marcado (Mark)**: se marcan los objetos alcanzables desde las raíces.
2. **Barrido (Sweep)**: se libera la memoria de los objetos no marcados.
3. **Compactación**: se reorganizan los objetos supervivientes.
4. **Recolección generacional**: Young Generation (Scavenge) y Old Generation (Mark-Sweep-Compact).

C Anexo C: Fuentes Consultadas

- MDN Web Docs – Gestión de Memoria en JavaScript.
- Documentación oficial del motor V8.
- Sílabo de Estructura de Datos – Universidad Nacional de Loja (2026).

— *Fin del documento* —